

Deep Learning (IST, 2024-25)

Homework 2

Group 52

Gonalo Rua (102604), Joo Gouveia (102611)

All the work was carried out by both members of the group, always together when trying to solve the problems presented in the different questions (including the ones that involved programming), and discussing the answer for them. The training of the neural networks was done in Gonalo's computer with a GTX 1070, which helped speed up the process. For this the flag **-device cuda** was used in the training of the models, after installing the necessary dependencies on the machine, which included the NVIDIA CUDA drivers for Arch Linux¹.

Question 1

1.

$$\begin{aligned}
 \textcircled{1} \quad E(q) &= \underbrace{-\text{lse}(\beta, Xq)}_{E_2(q)} + \underbrace{\beta^{-1} \log N + \frac{1}{2} q^T q + \frac{1}{2} M^2}_{E_1(q)} \\
 E_1(q) &= \beta^{-1} \log N + \frac{1}{2} q^T q + \frac{1}{2} M^2 = \frac{1}{2} q^T q + \beta^{-1} \log N + \frac{1}{2} M^2 \\
 E_2(q) &= -\text{lse}(\beta, Xq) \\
 \nabla E_1(q) &= \frac{\partial}{\partial q} \left(\beta^{-1} \log N + \frac{1}{2} q^T q + \frac{1}{2} M^2 \right) = \frac{\partial}{\partial q} \left(\frac{1}{2} q^T q \right) = \\
 &= q, \text{ because } \frac{\partial}{\partial q} \left(\frac{1}{2} q^T q \right) = \frac{\partial}{\partial q} \left(\frac{1}{2} \sum_{i=1}^D q_i^2 \right) \\
 \nabla E_2(q) &= \frac{\partial}{\partial q} \left(-\text{lse}(\beta, Xq) \right) = \frac{\partial}{\partial q} \left(-\beta^{-1} \log \sum_{i=1}^N \exp(\beta z_i) \right) = \\
 \Leftrightarrow \nabla E_2(q) &= \frac{\partial}{\partial q} \left(\beta^{-1} \log \sum_{i=1}^N \exp(\beta z_i) \right) = \\
 &= \beta^{-1} \cdot \frac{1}{\sum_{i=1}^N \exp(\beta z_i)} \nabla \left(\sum_{i=1}^N \exp(\beta z_i) \right) = * \\
 \nabla \left(\sum_{i=1}^N \exp(\beta z_i) \right) &= \sum_{i=1}^N \nabla \exp(\beta z_i) = \sum_{i=1}^N \exp(\beta z_i) \cdot \nabla(\beta z_i) = \\
 &= \sum_{i=1}^N \exp(\beta z_i) \cdot \nabla(\beta Xq)_i = \sum_{i=1}^N \exp(\beta z_i) \beta X_i^T \\
 * &= \beta \frac{\sum_{i=1}^N \exp(\beta z_i) X_i^T}{\sum_{i=1}^N \exp(\beta z_i)} = \sum_{i=1}^N \text{softmax}(\beta z)_i X_i^T = \\
 &= X^T \text{softmax}(\beta Xq) \quad \text{softmax}(\beta z)_i = \frac{\exp(\beta z_i)}{\sum_{j=1}^N \exp(\beta z_j)} \\
 &= \nabla E_2(-q)
 \end{aligned}$$

1 <https://wiki.archlinux.org/title/GPGPU#CUDA>

$$H_{E_1} = \frac{\partial^2}{\partial q^2} \left(\frac{1}{2} q^T q \right) = \frac{\partial^2}{\partial q} (q) = I$$

↑
identity matrix is positive semi-definite (hint 2)

$$H_{E_2} = \frac{\partial^2}{\partial q} (x^T \text{softmax}(\beta x q)) = \beta x^T (\text{diag}(p) - p p^T) x$$

↑
 $p = \text{softmax}(\beta x q)$ covariance matrix of softmax probabilities
(covariance matrices are always positive semi-definite) * 1

H_{E_2} is also positive semi-definite, since:

- $\text{diag}(\text{softmax}(\beta x q)) - \text{softmax}(\beta x q) \text{softmax}(\beta x q)^T$ is positive semi-definite * 1
- $\beta > 0$

2.

$$\tilde{E}_2(q) = E_2(q_t) + (\nabla E_2(q_t))^T (q - q_t)$$

$$\nabla(-E_2(q)) = x^T \text{softmax}(\beta x q) \Rightarrow \nabla(E_2(q)) = -x^T \text{softmax}(\beta x q)$$

$$\tilde{E}_2(q) = E_2(q_t) + (-x^T \text{softmax}(\beta x q_t))^T q - (-x^T \text{softmax}(\beta x q_t))^T q_t$$

these ~~three~~ ^{four} terms will be ignored on the optimization since they ~~are constants~~ ^{are constants (do not depend on q)}

$$E_1(q) = \frac{1}{2} q^T q + \beta^{-1} \log N + \frac{1}{2} M^2$$

$$q_{t+1} = \arg \min_q E_1(q) + \tilde{E}_2(q) = \arg \min_q \left(\frac{1}{2} q^T q + (-x^T \text{softmax}(\beta x q_t))^T q \right)$$

Finding the minimum:

$$\nabla \left(\frac{1}{2} q^T q + (-x^T \text{softmax}(\beta x q_t))^T q \right) = 0 \Rightarrow$$

$$\Rightarrow q - x^T \text{softmax}(\beta x q_t) = 0 \Rightarrow q = x^T \text{softmax}(\beta x q_t)$$

So,

$$q_{t+1} = x^T \text{softmax}(\beta x q_t)$$

3.

③

The cross-attention layer works by:

1. Calculating the similarity scores $= QK^T = q_t^T X^T$,
~~query~~ which are scaled by $\beta = \frac{1}{\sqrt{D}}$, so scaled scores $= \frac{1}{\sqrt{D}} QK^T = \frac{1}{\sqrt{D}} q_t^T X^T$
2. Apply softmax to normalize the attention weights $= \text{softmax}(\frac{1}{\sqrt{D}} QK^T)$
3. output = attention weights $\cdot V = \text{softmax}(\frac{1}{\sqrt{D}} QK^T) V = \text{softmax}(\frac{1}{\sqrt{D}} QK^T) X$

The Hopfield update rule is:

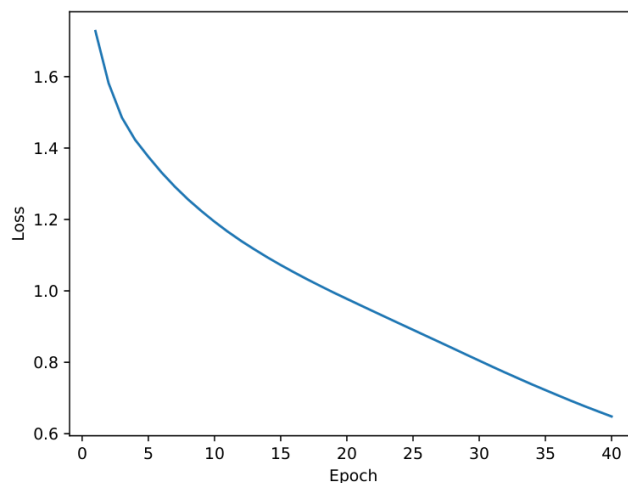
$$q_{t+1} = X^T \text{softmax}(\beta X q_t), \text{ so with } \beta = \frac{1}{\sqrt{D}} \text{ we have:}$$

$$q_{t+1} = X^T \text{softmax}(\frac{1}{\sqrt{D}} X q_t)$$

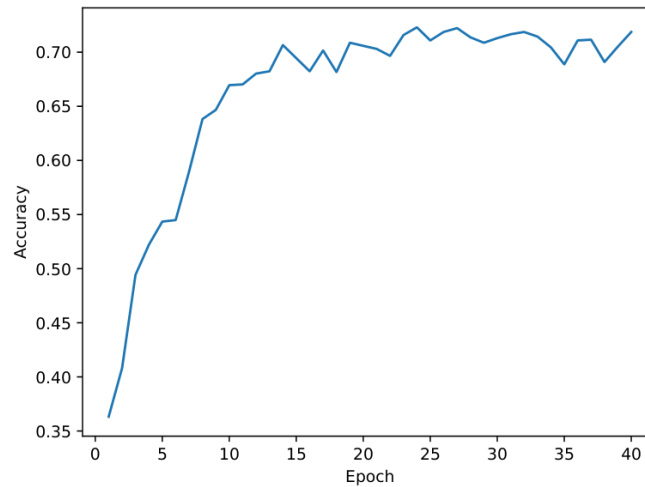
R.i Both methods are similar in the sense that they both compute ~~the~~ similarity scores scaled by $\beta = \frac{1}{\sqrt{D}}$, and they use Xq_t (for the Hopfield method) or QK^T (for the transformer method) to compute the normalized attention weights. The main difference between the two methods is that while the Hopfield network updates involve only one query vector q_t and compute q_{t+1} as a weighted sum of rows of X , the transformers process multiple queries (the rows of Q) simultaneously, which results in a matrix with each row corresponding to the weighted sum of a specific query.

Question 2

1. We implemented all the requirements for this exercise, and, after training the model for 40 epochs using SGD with learning rate 0.1, 0.01, and 0.001, the best configuration achieved was with learning rate 0.01. The obtained plot representing the training loss as a function of the epoch number was the following:

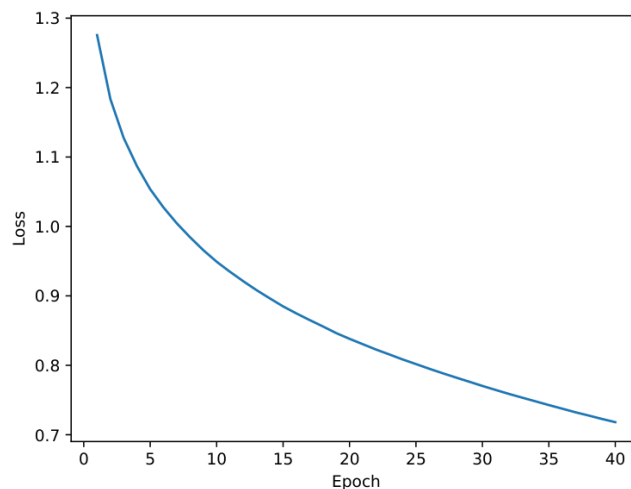


And the obtained plot representing the validation accuracy as a function of the epoch number was the following:

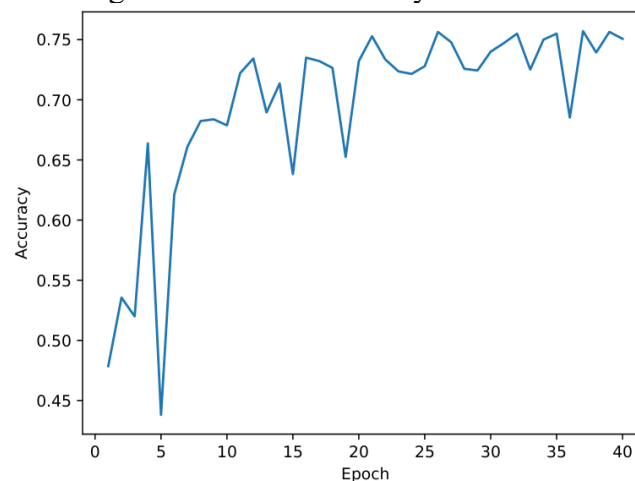


The final test accuracy obtained was 0.6960, and the final validation accuracy obtained was 0.7187.

2. We implemented the changes described, and, after training the model for 40 epochs using SGD with learning rate 0.01, we obtained the following plot representing the training loss as a function of the epoch number:



And the following plot representing the validation accuracy as a function of the epoch number:



We see an increased performance after the implementation of batch normalization in each convolutional block and in the MLP block, and the replacement of flattening the output of the convolutional blocks with a global average pooling layer. The final test accuracy increased from 0.6960 to 0.7403. The final validation accuracy increased from 0.7187 to 0.7507.

3. After implementing the function, we got 5340742 as the total number of trainable parameters for the first version of the network, and 755718 number of trainable parameters for the second version. The replacement of a flattened input to the fully connected layers with global average pooling drastically reduces the number the number of weights in the first FC layer (reduced from $\text{channels}[3]*6*6$ to $\text{channels}[3]$). Although the introduction of batch normalization introduces new parameters, these are small in comparison to the reduction of the input of the first FC layer.

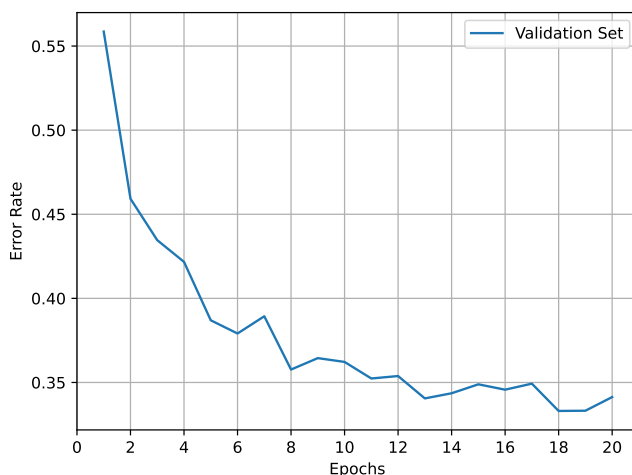
The second version presents better performance for various reasons. Firstly, the usage of a smaller number of parameters reduces the chance of overfitting, when compared with the first version. Secondly, the usage of global average pooling also helps in summarizing spacial features, rather than flattening them, which improves the generalization of the model. Finally, batch normalization in the second version can help increase the speed and stabilize the training.

4. We use small kernels in convolutional layers because they reduce the number of parameters compared to larger kernels, which makes the network faster and easier to train. Another important reason is that smaller kernels help the extraction of fine-grained features (e.g., edges, corner, etc), which can be combined in deeper layers to form more complex patterns. Larger kernels don't give us these fine-grained details and focus more on broader patterns.

Pooling layers are used because they help retain important information from the feature maps while reducing their dimensions. They aggregate the features, summarizing the feature maps and thus making it easier for the network to learn high-level features from low-level ones. This also gives the model some translation invariance, which means, depending on the magnitude, some translations of an object will not significantly change the output.

Question 3

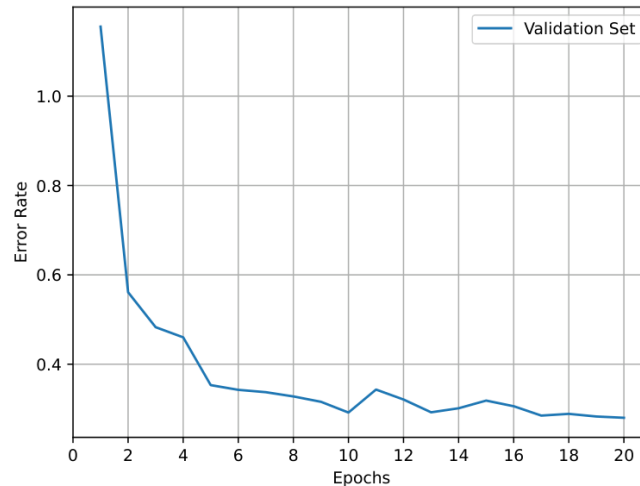
1. a) After training the model for 20 epochs using a learning rate of 0.003, a dropout rate of 0.3, a hidden size of 128, and a batch size of 64, we obtained the following plot of the validation CER at the end of each epoch:



And after running the model checkpoint on the test set we obtained:

Test CER: 0.3353, Test WER: 0.8290

1. b) After training the model with the same hyperparameters as 1. a), but this time with the `--use_tan` flag, we obtained the following plot of the validation CER at the end of each epoch:



And after running the model checkpoint on the test set we obtained:

Test CER: 0.2349, Test WER: 0.7460

1. c) We implemented the `nucleus_sampling` function and ran the command:

```
$ python hw2-q3.py test --use_attn --checkpoint_name attn.pt --topp 0.8
```

This gave us the following output:

Test CER: 0.2421, Test WER: 0.7620

Printing first 10 examples with multiple predictions:

```
('remainder', {'romainder', 'remainder'})  
( 'crudités', {'cruditate', 'cruditay'})  
( 'Jingchuan', {'jinchuan', 'jinchau', 'jinchhean'})  
( 'glossal', {'glossel', 'glossal', 'glossul'})  
( 'companions', {'companunnese', 'companunnesees', 'companiians'})  
( 'slaggy', {'slagy', 'slaggy'})  
( 'Bronx', {'brongs', 'brongs'})  
( 'Hebrides', {'hebrades', 'hebrodies'})  
( 'sternutatory', {'strnutatory', 'stroututory', 'strutotorary'})  
( 'consubstantiation', {'consubstiaciation', 'consubstiatiation'})
```

Test WER@3: 0.7170